

E-Grocery Defense Script (REVIEWER 2.0)

How to use this: Open your app, walk through each feature, and read the talking points aloud. Each section tells you **what to say** and **what to point at** on screen.

Opening Line

"Good morning/afternoon, Professor. This is E-Grocery ni Aling Bebang, an ASP.NET Core MVC web application. It uses the Model-View-Controller pattern, Entity Framework Core for the database, and Razor views for the user interface. I'll now demonstrate the features."

Demo 1: Show the Architecture

Point to: `E-Grocery` solution folder in Visual Studio

Say:

- **Model** — our data classes: `User`, `Product`, `CartItem`, `Order`
- **View** — our `.cshtml` files in the `Views` folder
- **Controller** — `AccountController`, `StoreController`, `AdminController`
- **DbContext** — `AppDbContext.cs` bridges our C# code to SQL Server

Key line: *"When a user visits a page, the Controller receives the request, uses the Model to get or save data, then returns a View to display the result."*

Demo 2: Show the Database

Point to: `appsettings.json` → connection string

Say:

"Our database is SQL Server LocalDB. This connection string tells EF Core where to find it. We didn't write SQL by hand — we used Migrations. We run `Add-Migration` to create the schema and `Update-Database` to build the tables."

Point to: `AppDbContext.cs`

Say:

"This is our DbContext. It has four `DbSet` properties — one for each table: `Users`, `Products`, `CartItems`, and `Orders`. EF Core tracks these objects and generates the SQL for us."

Demo 3: Register an Account

Action: Click Register, fill the form, then submit with an empty field first.

Say (before clicking):

"Before I click Create Account, watch what happens. Our app has two layers of validation. First, the browser checks the form using jQuery Validation — this is client-side validation."

Action: Leave a field blank and click Submit. Red errors appear instantly.

Say:

"See? The error appeared immediately without sending anything to the server. The browser blocked the submit."

Action: Now fill all fields correctly and click Submit.

Say:

"Now the data travels to the server. ASP.NET uses Model Binding to convert the form values into a `RegisterViewModel` object. Then `ModelState.IsValid` checks the validation rules — `[Required]`, `[EmailAddress]`, `[StringLength]`, `[Compare]` — on the server side. Only if both layers pass does the database code run."

Point to: `AccountController.cs` → `Register` POST action

Say:

"Here in the controller: if `ModelState.IsValid` is true, we create a `User` object, add it to `_context.Users`, and call `SaveChanges()`. This is the only point where SQL INSERT runs. If validation fails, we hit `return View(model)` immediately — the database is never touched."

Key defense point:

"So validation ALWAYS comes before the database. Invalid data cannot reach SQL Server."

Demo 4: Login

Action: Log in with the account you just created.

Say:

"When login succeeds, we store the user's email in Session. Session remembers the user across page visits. We check it in `_Layout.cshtml` to show 'My Account' instead of 'Login'."

Point to: `_Layout.cshtml` → session check

Say:

"Here we get the session value. If it exists, we show My Account and Logout. If not, we show Login and Register."

Demo 5: Browse Products

Action: Go to the store homepage.

Say:

"This is the Store page. The `StoreController.Index()` action fetches all products using `_context.Products.ToList()` and passes them to the view. The view loops through them with `@foreach` and displays each card."

Point to: `StoreController.cs` → `Index()`

Say:

"We use `ToList()` to get all records, then `foreach` to display them. This follows our class notes exactly."

Demo 6: Search Products

Action: Type "apple" in the search bar.

Say:

"Search uses `method="get"` so the term appears in the URL. The controller receives it through Model Binding, gets all products with `ToList()`, then loops with `foreach` to find matches. It reuses the same `Index.cshtml` view with the filtered list."

Point to: `StoreController.cs` → `Search(string searchTerm)`

Demo 7: Add to Cart

Action: Click a product, pick a quantity, click Add to Cart.

Say:

"When I click Add to Cart, the form POSTs the `productId` and `quantity` to `StoreController.AddToCart()`. The controller finds the product, checks if it's already in the cart, and either updates the quantity or creates a new `CartItem`. Cart data lives in the database, not the browser, so it survives page refreshes."

Point to: `StoreController.cs` → `AddToCart()`

Demo 8: View Cart

Action: Click the cart icon.

Say:

"The cart page gets all `CartItems` from the database, calculates the total using a `foreach` loop, and passes it to the view via `ViewData['Total']`. The view displays each item with its subtotal."

Point to: `StoreController.cs` → `Cart()`

Demo 9: Place Order

Action: Click Place Order.

Say:

"Placing an order does two things: first, it creates an *Order* record for every item in the cart. Second, it deletes all *CartItem* records to clear the cart. Both operations are handled by a single *SaveChanges()* call — EF Core generates the SQL INSERTs and DELETEs in one transaction."

Point to: *StoreController.cs* → *PlaceOrder()*

Demo 10: Order History

Action: Click My Account → Order History.

Say:

"Order history gets the user's email from *Session*, fetches all orders with *ToList()*, then filters with *foreach* to show only this user's records."

Demo 11: Profile Picture Upload

Action: Go to My Account, change the name, upload a picture, save.

Say:

"The form uses *enctype="multipart/form-data"* because it includes a file. The controller accepts *IFormFile*. We extract the file extension, generate a unique filename using *DateTime.Now.Ticks*, save it to *wwwroot/images/profiles/* using a *FileStream* inside a *using* block, and store only the file path in the database."

Point to: *AccountController.cs* → *Profile()* POST

Say:

"The *using* block automatically closes the file after saving, which frees memory and prevents the file from staying locked. The actual image is on the server disk; the database only stores the path."

Demo 12: Admin — Create Product

Action: Go to */Admin/Create*, fill the form, upload a product image, click Save.

Say:

"The admin page uses the same *ProductCreateViewModel* for both creating and editing. When the admin clicks Save, *ModelState.IsValid* runs first. If valid, we check if an image was uploaded, save it with the same *DateTime.Now.Ticks* pattern, and either insert a new product or update an existing one depending on whether *model.Id* is zero or not."

Point to: *AdminController.cs* → *Create()* POST

Demo 13: Admin — Edit & Delete

Action: Click Edit on a product, change the price, save. Then click Delete.

Say:

"Edit reuses the same Create view. The `Edit(int id)` action finds the product, copies its data into a `ViewModel`, and returns `View('Create', model)` so the form is pre-filled. After saving, `model.Id > 0` tells the controller to update instead of insert. Delete uses a confirmation modal, then removes the product from the database with `Remove()` and `SaveChanges()`."

The Golden Rule — Say This When Asked "What Triggers First?"

Memorize this exact order:

1. **Client-Side Validation** — browser checks form rules instantly
2. **Model Binding** — ASP.NET converts form data to a C# object
3. **Server-Side Validation** — `ModelState.IsValid` checks rules again
4. **Database Operations** — only if validation passed
5. **SaveChanges()** — EF Core runs SQL
6. **Redirect** — sends user to a new page

One-liner: "Validation comes before the database. Every time."

Quick Reference: One-Liner Definitions

Term	One-Liner
MVC	Model (data), View (UI), Controller (requests)
EF Core	ORM — C# objects become SQL automatically
DbContext	Tracks objects and generates SQL on <code>SaveChanges()</code>
DbSet	Represents one database table inside <code>DbContext</code>
Model Binding	ASP.NET auto-fills <code>ViewModel</code> from form data
ModelState.IsValid	Security guard — checks rules before DB code
ViewModel	Form-only class with validation (not a DB table)
Data Annotations	<code>[Required]</code> , <code>[EmailAddress]</code> — validation rules
Session	Remembers the logged-in user across pages
Migration	C# file that builds database tables via <code>Update-Database</code>
Dependency Injection	ASP.NET auto-gives us <code>AppDbContext</code> in the constructor
Razor	@ syntax that mixes C# into HTML views
Tag Helper	<code>asp-for</code> , <code>asp-action</code> — auto-generates form HTML

Term	One-Liner
IFormFile	Represents an uploaded image/file in the controller
FileStream	Opens a file on disk to read or write
RedirectToAction	Sends browser to a new page (prevents duplicate submits)
GET	Shows a page — safe, read-only
POST	Submits data — changes the database
ToList()	Gets all records from a table
foreach	Loops through records one by one
wwwroot	Public folder — images, CSS, JS live here
using	Auto-closes a resource like FileStream
Ticks	Unique number from <code>DateTime.Now</code> for filenames

If the Professor Asks Something Unexpected

Fallback strategy: Point to `REVIEWER.md` and say:

"That is covered in detail in our technical documentation. The short answer is..." — then pick the closest one-liner from the table above.

If asked about a specific file:

"Let me show you the code." — open the file and read the relevant `if` block or `foreach` loop aloud.

If asked why you didn't use `async` or LINQ:

"Our class notes teach synchronous `IActionResult` with `ToList()` and `foreach` loops. We followed exactly what was taught."

Closing Line

"That concludes the demonstration of E-Grocery ni Aling Bebang. Thank you, Professor."